

Coaching Tool for Public Engagement

Technical Architecture & System Prompt Reference

March 2026

This document covers the coaching tool's architecture, current deployment on Render.com, proposed AWS migration using **ECS Fargate + Lambda** (hybrid approach), data pipeline, agent system, and all system prompts powering LLM interactions.

Contents

1. System Overview
2. Current Architecture
3. Tech Stack
4. Current Deployment (Render.com)
5. Proposed AWS Architecture (ECS Fargate + Lambda)
6. Data Pipeline (Current)
7. Data Pipeline on AWS (Step Functions + Lambda)
8. Agent System & Tool Calling
9. Frontend Pages & Routes
10. API Endpoints
11. System Prompts – Detailed Reference
12. Environment Variables

1. System Overview

The Coaching Tool for Public Engagement is a RAG (Retrieval-Augmented Generation) application that helps practitioners plan, assess, and improve their community engagement projects. It combines a curated knowledge base of engagement guides, case studies, and best practices with an agentic LLM system that searches this knowledge base before every recommendation.

The tool offers two main user journeys: (1) a Nesta Framework Self-Assessment with AI coaching for each of 9 dimensions, and (2) a Case Study Library that scores real-world case studies for relevance to the user's situation. A general-purpose Chatbot is also available for ad-hoc questions about public engagement.

2. Current Architecture

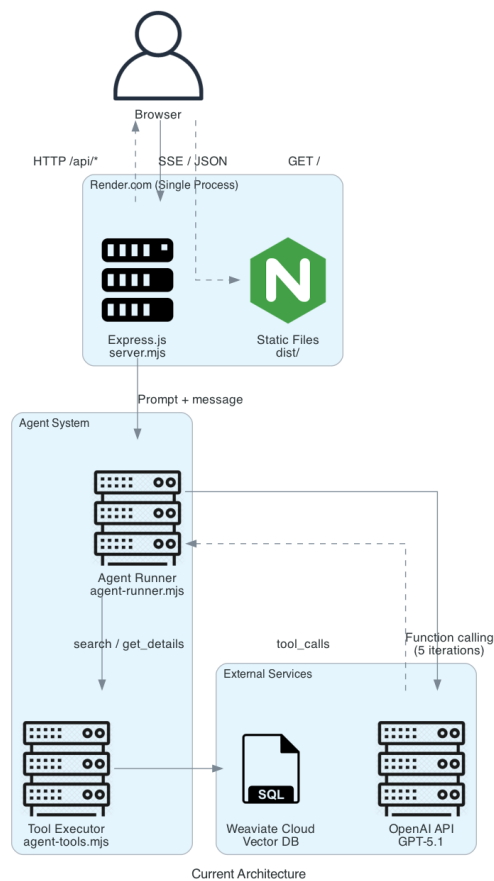


Figure 1: Current system architecture

3. Tech Stack

Layer	Technology	Details
Frontend	React 18 + Vite 6 + TypeScript	SPA with react-router v7, Tailwind CSS v4, Radix UI
Backend	Express.js 5 (Node 20+)	ES Modules, SSE streaming, agent loop
LLM	OpenAI GPT-5.1	Chat completions + function calling; mini for classification
Vector DB	Weaviate Cloud	text2vec-openai, BM25 + nearText hybrid search
Deployment	Render.com (current)	Auto-deploy from GitHub, free tier

Layer	Technology	Details
Testing	Vitest + Testing Library	Unit tests for React components

4. Current Deployment (Render.com)

The app runs as a single web service on Render.com's free tier, configured via `render.yaml`. Vite builds the frontend to `dist/`, and Express serves both the static SPA and the API from one process.

Setting	Value
Service type	Web service (Node.js)
Plan	Free tier
Auto-deploy	Yes (push to main)
Build	<code>npm install && npm run build (vite build)</code>
Start	<code>node server.mjs</code>
Node	20.12.2

Limitations: Free tier has cold starts, 512 MB RAM, no auto-scaling, and the server spins down after 15 min of inactivity.

5. Proposed AWS Architecture (ECS Fargate + Lambda)

The proposed migration uses a **hybrid approach**: ECS Fargate runs the Express.js backend as a Docker container behind an Application Load Balancer, while Lambda functions handle the document ingestion pipeline and async evaluation jobs. This approach is cleaner because ECS natively supports long-lived HTTP connections and SSE streaming without any workarounds.

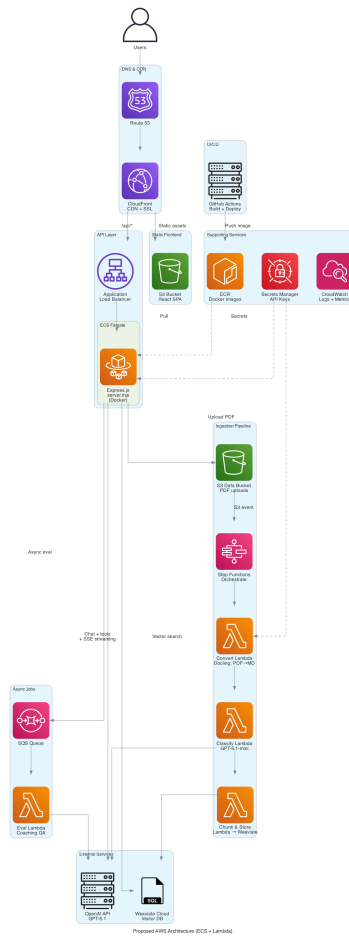


Figure 2: Proposed AWS architecture – ECS Fargate + Lambda hybrid

Component Breakdown

AWS Service	Role	Why
S3 + CloudFront	Static frontend + CDN	Global edge caching, HTTPS, custom domain. Serves React SPA independently from the API.
Route 53	DNS management	Custom domain routing to CloudFront distribution and ALB.
ALB	Load balancer	Routes /api/* requests to ECS tasks. Handles health checks, SSL termination, and path-based routing.
ECS Fargate	Express.js backend	Runs the Express server as a Docker container. Natively supports SSE streaming, WebSockets, and long-lived connections. Auto-scales based on CPU/memory. No server management.
ECR	Docker image registry	Stores the Express.js Docker image. GitHub Actions pushes new images on merge to main.
Step Functions	Ingestion orchestration	Orchestrates the 3-step document ingestion pipeline: Convert → Classify → Chunk & Store. Handles retries, error states, and parallel document processing.
Lambda (Convert)	PDF → Markdown via Docling	Python Lambda with Docling. Converts PDFs to structured Markdown preserving headings, tables, layout.
Lambda (Classify)	Document classification	Calls GPT-5.1-mini to auto-classify documents (content_type + summary).

AWS Service	Role	Why
Lambda (Chunk & Store)	Chunking + Weaviate storage	Heading-based splitting (#####), 300-char overlap, contextual prefix. Stores in Weaviate.
Lambda (Eval)	Coaching QA evaluation	Triggered by SQS after each coaching message. Runs evaluate-coaching.txt, logs scores.
S3 (Data Bucket)	PDF storage	Stores uploaded PDFs. S3 events trigger the Step Functions ingestion pipeline.
SQS	Async job queue	Decouples coaching eval from the API. ECS sends a message, Eval Lambda picks it up.
Secrets Manager	API key storage	OPENAI_API_KEY, WEAVIATE_API_KEY. Injected into ECS tasks and all Lambdas at runtime.
CloudWatch	Monitoring + logs	ECS task logs, per-Lambda logs, invocation metrics, error rate alarms, coaching-eval score tracking.

Why ECS Fargate for the API Server?

- SSE streaming works natively – Express's `res.write()` sends data over a persistent HTTP connection. No Lambda streaming workarounds needed.
- Same Express codebase – Zero code changes. The Dockerfile just does `npm install && node server.mjs`.
- Long-running requests – Agent loops with multiple tool calls can take 30-60 seconds. ECS has no timeout constraints (unlike Lambda's 15-min max and API Gateway's 30-sec integration timeout).
- WebSocket-ready – If future features need WebSocket support, ECS handles it natively.
- Predictable performance – No cold starts after the first deployment. Fargate tasks stay warm as long as there's an active deployment.

Why Lambda for the Pipeline?

- Event-driven – Ingestion runs only when documents are uploaded. No need for always-on compute.
- Isolated runtimes – The Convert Lambda uses Python 3.12 + Docling; other Lambdas use Node.js 20. Separate runtimes without multi-container complexity.
- Cost-efficient – Pipeline runs infrequently. Lambda charges per invocation; zero cost when idle.
- Step Functions orchestration – Built-in retry logic, error handling, and parallel processing. Visual workflow monitoring in the AWS console.

Migration Path

- **Phase 1:** Dockerize Express app. Push image to ECR. Create ECS Fargate service + ALB. Test all endpoints.
- **Phase 2:** Move frontend to S3 + CloudFront. Configure CloudFront to route `/api/*` to ALB origin.
- **Phase 3:** Create 3 pipeline Lambdas (Convert, Classify, Chunk & Store) + Step Functions state machine. Create S3 data bucket with event trigger.
- **Phase 4:** Create SQS queue + Eval Lambda. Wire ECS to send eval jobs to SQS.
- **Phase 5:** DNS cutover via Route 53. Secrets migration to Secrets Manager. CloudWatch dashboards.
- **Phase 6 (optional):** Migrate Weaviate to self-hosted for cost control at scale.

Cost Estimate (Low Traffic)

Service	Estimated Monthly Cost
ECS Fargate (0.25 vCPU, 512 MB)	~\$9–12/month (always-on single task)
ALB	~\$16/month (fixed) + \$0.008/LCU-hour
Pipeline Lambdas (3)	Free tier: 1M requests + 400K GB-s
Eval Lambda	Included in Lambda free tier

Service	Estimated Monthly Cost
Step Functions	Free tier: 4000 state transitions/mo
SQS	Free tier: 1M requests
S3 (data + frontend)	< \$1
CloudFront	< \$1 (low traffic)
ECR	~\$0.10/GB (one image ≈ 200 MB)
Secrets Manager	~\$1
CloudWatch	Free tier likely sufficient
Total	~\$27–32/month

Note: Weaviate Cloud and OpenAI API costs are separate. The ECS approach costs more than fully serverless (~\$3–5/month) but provides simpler SSE streaming, no cold starts, and a more traditional deployment model. For scale-up, ECS auto-scaling can add Fargate tasks based on CPU/memory thresholds.

6. Data Pipeline (Current)

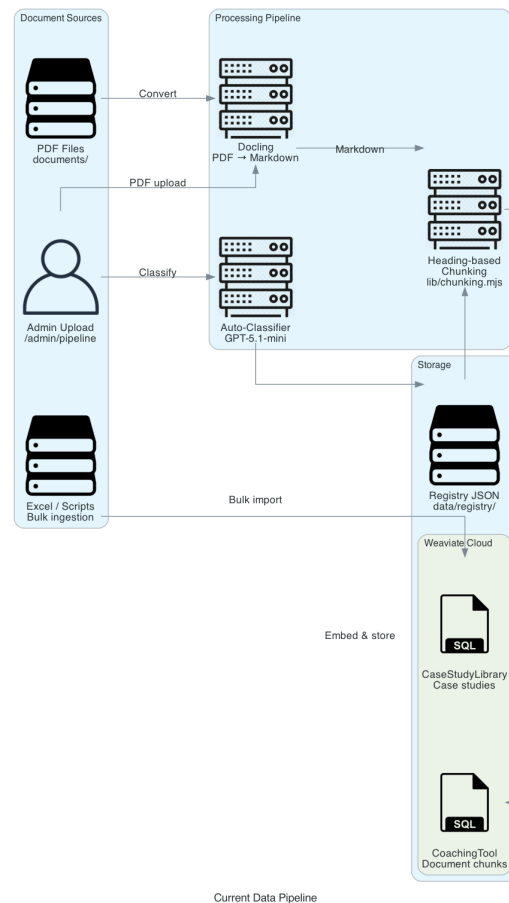


Figure 3: Current data pipeline

Path A: PDF Ingestion (CLI)

PDFs in documents/ are first converted to structured Markdown using Docling, then chunked via heading-based splitting. Stored in Weaviate CoachingTool collection.

- PDF → Markdown – scripts/convert-pdfs.py runs Docling (DocumentConverter) to convert PDFs to structured Markdown preserving headings, tables, and layout.
- Heading-based chunking – lib/chunking.mjs parses Markdown headings (####/####) and splits into semantic sections (max ~2000 chars).
- Overlap & context – ~300 char overlap between chunks + contextual retrieval prefix (document title + section path).
- Embedding & storage – Chunks embedded via text2vec-openai and stored in Weaviate.

Path B: Admin Dashboard Upload

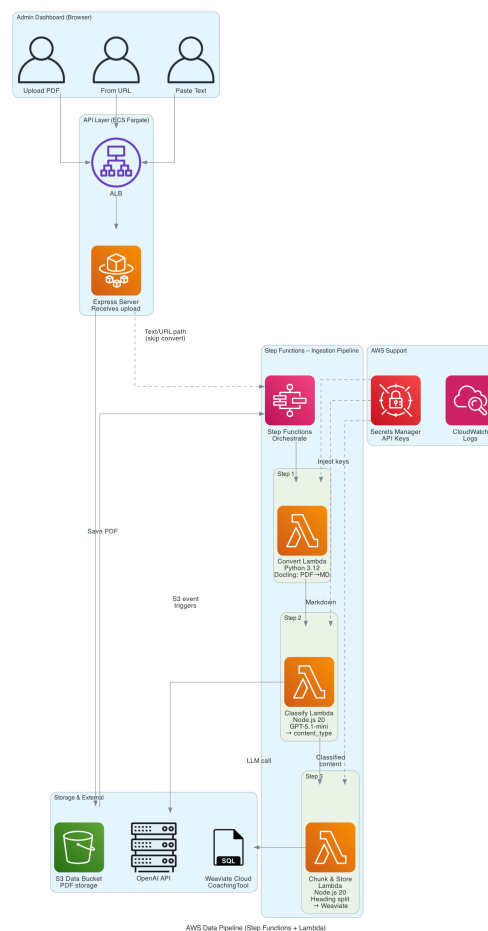
Admin UI at /admin/pipeline for uploading documents. Auto-classified using the classify-document.txt prompt (GPT-5.1-mini). Registry JSON stored in data/registry/.

Weaviate Collections

Collection	Purpose	Key Fields
CoachingTool	Document chunks	content, document_id, doc_name, source_url, content_type, section_name, chunk_index
CaseStudyLibrary	Case studies	document_id, title, summary, location, timeframe, demographic, scale, tags, key_outcomes

7. Data Pipeline on AWS (Step Functions + Lambda)

On AWS, document ingestion runs as a Step Functions state machine that orchestrates 3 Lambda functions in sequence. The pipeline is triggered when a PDF is uploaded to the S3 data bucket (via admin dashboard through ECS, or GitHub Actions sync). For text/URL ingestion, the Convert step is skipped.



Step Functions Pipeline

The state machine runs 3 steps in sequence. Each step is a separate Lambda function. If any step fails, Step Functions retries it (configurable) and logs the error to CloudWatch.

Step 1: Convert Lambda (Python 3.12)

- Triggered by S3 event when a PDF is uploaded to the data bucket.
- Downloads the PDF from S3.
- Runs Docling (`docling.document_converter.DocumentConverter`) to convert PDF to structured Markdown.
- Docling preserves document structure: headings, tables, figures, lists, and page layout.
- Outputs the Markdown content to the next step.
- Skipped for text paste and HTML URL paths – content goes directly to the Classify step.
- Memory: 1024 MB | Timeout: 5 min.

Step 2: Classify Lambda (Node.js 20)

- Receives Markdown content (from Convert step or directly from the ECS API for text/URL paths).
- Calls GPT-5.1-mini with the `classify-document.txt` prompt (temp 0).
- Returns `{content_type, summary}` – one of: `case_study`, `transcript`, `blog_post`, `journal_article`, `report`, `guide`, `policy_brief`, `lecture`, `tool_or_resource`, `other`.
- Memory: 256 MB | Timeout: 30s.

Step 3: Chunk & Store Lambda (Node.js 20)

- Receives classified content + metadata from the Classify step.
- Runs heading-based chunking (same logic as `lib/chunking.mjs`):
 - Parses Markdown headings (`#`, `##`, `###`, `####`) to identify semantic sections.
 - Splits on heading boundaries into coherent chunks (max ~2000 chars).
 - Sub-splits oversized sections at paragraph boundaries.
 - Adds ~300 char overlap between consecutive chunks.
 - Prepends a contextual retrieval prefix (document title + section path) to each chunk.
- Chunks are sent to Weaviate Cloud, where `text2vec-openai` generates embeddings.
- Stored in `CoachingTool` collection with full metadata.
- Memory: 512 MB | Timeout: 5 min.

Coaching Eval (ECS → SQS → Eval Lambda)

- After each coaching message, the ECS API server sends a job to an SQS queue.
- Eval Lambda polls the queue, runs `evaluate-coaching.txt` (GPT-5.1, temp 0). Scores logged to CloudWatch.
- Fully async – the user never waits. Decouples QA from the chat response.
- Memory: 256 MB | Timeout: 1 min.

Lambda Function Summary

Lambda	Runtime	Memory	Timeout	Trigger	What It Does
Convert	Python 3.12	1024 MB	5 min	Step Functions	Docling: PDF → structured Markdown
Classify	Node.js 20	256 MB	30s	Step Functions	GPT-5.1-mini: content_type + summary
Chunk & Store	Node.js 20	512 MB	5 min	Step Functions	Heading-based split → Weaviate

Lambda	Runtime	Memory	Timeout	Trigger	What It Does
Eval	Node.js 20	256 MB	1 min	SQS queue	Coaching QA scores → CloudWatch

Key Differences from Current Setup

Aspect	Current (Render / Local)	AWS (ECS + Lambda)
API server	Express on Render (always-on process)	ECS Fargate – containerized Express, auto-scaling, ALB
SSE streaming	Express SSE (long-lived process)	ECS – native SSE over persistent HTTP (no workarounds)
PDF conversion	Docling runs in Express (blocks server)	Convert Lambda – isolated Python 3.12 with Docling
Ingestion pipeline	Manual CLI scripts	Step Functions: Convert → Classify → Chunk & Store
Coaching eval	Fire-and-forget in Express	SQS → Eval Lambda – fully decoupled
Secrets	Env vars in Render dashboard	Secrets Manager – injected at runtime, rotatable
Monitoring	Console logs only	CloudWatch – ECS + per-Lambda, searchable, alertable
Cost when idle	Free (Render free tier spins down)	~\$25–28/month (ECS always-on + ALB fixed cost)

8. Agent System & Tool Calling

Most prompts are agentic – they use OpenAI function calling to search Weaviate before generating responses, ensuring recommendations are grounded in actual documents.

- `runAgentLoop()` – Full loop, returns final text. Used by non-streaming endpoints.
- `resolveAgentToolCalls()` – Resolves tools, returns messages for streaming. Used by SSE endpoints.
- Up to 5 iterations. On the last, `tool_choice`: "none" forces text output.

Tool	Description	Parameters
<code>search_knowledge_base</code>	Semantic search over CoachingTool. BM25 first, <code>nearText</code> fallback.	<code>query</code> (required), <code>content_type_filter</code> (optional)
<code>get_document_details</code>	Retrieve all chunks for a document by ID.	<code>document_id</code> (required)

9. Frontend Pages & Routes

Route	Component	Description
<code>/</code>	Home	Landing page with nav cards to Coach and Case Studies
<code>/coach</code>	Coach	Nesta self-assessment: 9 free-text questions + scenario generator
<code>/coach/dashboard</code>	AssessmentDashboard	Status cards (addressed/partial/not-addressed) + coaching chat
<code>/coach/reflection</code>	Reflection	Final reflection after assessment + coaching
<code>/case-studies</code>	CaseStudies	Browse/search case study library with relevance scoring
<code>/case-studies/:id</code>	CaseStudyDetail	Individual case study detail view

Route	Component	Description
/admin	AdminDashboard	Knowledge base management dashboard
/admin/documents	DocumentExplorer	Browse documents in Weaviate
/admin/pipeline	PipelineManager	Upload, classify, and ingest documents

A general-purpose ChatBot component is accessible from every page. CoachingChatPanel is embedded in the AssessmentDashboard.

10. API Endpoints

Method	Endpoint	Prompt	Response
POST	/api/chatbot	chatbot.txt (+ coaching-context.txt)	SSE stream (markdown + sources)
POST	/api/evaluate-assessment	evaluate-assessment.txt	JSON {evaluations[]}
POST	/api/analyze-cross-resolution	cross-resolution.txt	JSON {currentQuestionResolved, resolvedQuestionIds[]}
POST	/api/generate-reflection	generate-reflection.txt	JSON {reflection{}}
POST	/api/score-case-studies	score-case-studies.txt	JSON {scores[]}
POST	/api/generate-scenario-responses	generate-scenario.txt	JSON {scenario, responses{}}
GET	/api/case-studies	(direct Weaviate query)	JSON array of case studies
POST	/api/admin/classify	classify-document.txt	JSON {content_type, summary}

On AWS: All endpoints route through ALB to the ECS Fargate Express server (including SSE chatbot). The evaluate-coaching.txt prompt runs via SQS → Eval Lambda.

11. System Prompts – Detailed Reference

All system prompts are stored as plain text files in the prompts/ folder and loaded at server startup via prompts/load.mjs. Prompts can be edited without touching any code – just modify the .txt file and restart/redeploy.

File	Constant Name	Prompt Type
chatbot.txt	CHATBOT_PROMPT	Agentic (tools + SSE streaming)
score-case-studies.txt	SCORE_CASE_STUDIES_PROMPT	Agentic (tools, JSON output)
evaluate-assessment.txt	EVALUATE_ASSESSMENT_PROMPT	Agentic (tools, JSON output)
coaching-context.txt	COACHING_CONTEXT_PREFIX	Template (client-side, injected into chatbot)
cross-resolution.txt	CROSS_RESOLUTION_PROMPT	Direct LLM call (no tools, JSON output)
generate-reflection.txt	GENERATE_REFLECTION_PROMPT	Agentic (tools, JSON output)
evaluate-coaching.txt	EVALUATE_COACHING_PROMPT	Direct LLM call (background, JSON output)
generate-scenario.txt	GENERATE_SCENARIO_PROMPT	Agentic (tools, JSON output)
scenario-descriptions.json	SCENARIO_DESCRIPTIONS	Data file (6 persona definitions)
classify-document.txt	CLASSIFY_SYSTEM	Direct LLM call (GPT-5.1-mini, JSON output)

11.1 Chatbot (General Assistant)

File: prompts/chatbot.txt | **Endpoint:** POST /api/chatbot | **Type:** Agentic + SSE | **AWS:** ECS Fargate

Purpose: General-purpose chatbot. Users ask any question about public engagement and get evidence-grounded answers. Also handles coaching chats when COACHING_CONTEXT_PREFIX is detected.

Input: User message + conversation history. Output: Streamed markdown with "### Sources" section.

Key Behaviors

- Must search KB at least once before answering
- Every recommendation cites source inline: [Source: Document Name]
- Admits gaps: "Based on available resources, I don't have specific guidance on this."

11.2 Score Case Studies (Relevance)

File: prompts/score-case-studies.txt | **Endpoint:** POST /api/score-case-studies | **Type:** Agentic + JSON | **AWS:** ECS Fargate

Purpose: Scores every case study in the library against the user's situation using a weighted 6-dimension rubric (100 points total).

Input: Practitioner context (10 fields) + list of case studies. Output: JSON scores array with dimension breakdown.

Scoring Rubric

Dimension	Max Points
Issue/Topic Alignment	20
Engagement Goal Match	20
Audience/Demographic Fit	15
Timeline & Scale Compatibility	15
Resource & Constraint Alignment	15
Stuck Point / Process Stage Relevance	15

Must search KB at least 4 times. Always returns at least 5 case studies sorted by score.

11.3 Evaluate Assessment (Nesta Framework)

File: prompts/evaluate-assessment.txt | **Endpoint:** POST /api/evaluate-assessment | **Type:** Agentic + JSON | **AWS:** ECS Fargate

Purpose: Evaluates practitioner's responses to 9 Nesta questions as "addressed," "partial," or "not-addressed." Gateway to coaching flow.

Input: 9 Nesta questions + user responses. Output: JSON with 9 evaluations (questionId, status, gap, coachingContext).

The 9 Nesta Questions

- Q1: Have you articulated the project's goals?
- Q2: Have you identified the right participants?
- Q3: Can you reach the participants you identified?
- Q4: Who is the right owner?
- Q5: Have you included incentives for participation?
- Q6: Have you defined the tasks?
- Q7: Have you established the workflow?
- Q8: How will you evaluate inputs?
- Q9: How will you use what the group creates?

Decision Test

- "addressed" – Names specific actions, people, methods, or timelines. Actionable without follow-up.
- "not-addressed" – Fails 2+ of: specific method? specific people? concrete step? on-topic?
- "partial" – Fails exactly 1 check. Does NOT default to partial for vague responses.

11.4 Coaching Context (Socratic Coach)

File: prompts/coaching-context.txt | **Used by:** CoachingChatPanel.tsx (client-side) | **Type:** Template

Purpose: Turns the general chatbot into a Socratic coach for a specific Nesta question. Injected as first message in coaching conversations.

Placeholders: {{question}}, {{userResponse}}, {{status}}, {{gap}}

Coaching Approach

- Starts by acknowledging what the user did well
- Socratic questioning – probing questions, not lectures
- Evaluates user proposals against the identified gap
- If proposal is strong, explicitly tells them to mark as resolved

11.5 Cross-Resolution Analyzer

File: prompts/cross-resolution.txt | **Endpoint:** POST /api/analyze-cross-resolution | **Type:** Direct LLM (no tools) | **AWS:** ECS Fargate

Purpose: After coaching about one Nesta question, determines if the conversation also resolved gaps from other unresolved questions. Prevents repetition across sessions.

Input: Transcript + current question gap + other unresolved cards. Output: JSON {currentQuestionResolved, resolvedQuestionIds[]}

Key Behaviors

- Deliberately conservative – requires concrete discussion that closes the gap
- Passing mentions and vague overlap do NOT count
- Only returns IDs of OTHER cards (not the current question)

11.6 Generate Reflection (Final Summary)

File: prompts/generate-reflection.txt | **Endpoint:** POST /api/generate-reflection | **Type:** Agentic + JSON | **AWS:** ECS Fargate

Purpose: Generates comprehensive final reflection after assessment + coaching, summarizing the journey, strengths, gaps, and priority next steps.

Input: All 9 responses + evaluations + coaching chat histories (classified as PRODUCTIVE / RESOLVED WITHOUT CONVERSATION / UNRESOLVED / NO SESSION).

Output: JSON reflection with: summary (2–3 paragraphs), addressed[], partial[] (with nextSteps), notAddressed[] (with nextSteps), priorityActions[3] (action, rationale, timeline).

Key Behaviors

- Accounts for full journey – original answers AND coaching conversations
- Acknowledges growth in productive conversations
- Flags when users skipped coaching despite gaps
- Exactly 3 priority actions with suggested timeframes

11.7 Evaluate Coaching (Background QA)

File: prompts/evaluate-coaching.txt | **Endpoint:** None (fire-and-forget; SQS → Eval Lambda on AWS) | **Type:** Direct LLM, temp 0

Purpose: Internal QA. Runs after every coaching message, logs results to console (or CloudWatch on AWS). User never sees this.

Input: Nesta question, original response, gap, full transcript. Output: JSON with 6 dimension scores (1–10), overall 0–100, strengths, weaknesses.

Dimensions: socratic_approach, specificity, evidence_use, warmth_and_tone, gap_targeting, progression.

11.8 Generate Scenario (Test Data)

File: prompts/generate-scenario.txt + scenario-descriptions.json | **Endpoint:** POST /api/generate-scenario-responses | **AWS:** ECS Fargate

Purpose: Testing tool. Generates realistic responses to 9 Nesta questions based on a persona scenario.

Input: Scenario key or custom description. Output: JSON with scenario label + 9 responses.

Built-in Personas

Scenario	Description
well-prepared	Thorough planner with clear goals and workflows
vague-minimal	Generic, minimal-effort responses
contradictory	Subtle internal contradictions between answers
off-topic	Confuses engagement with marketing; answers wrong questions
over-ambitious	Grand plans with minimal resources
hostile-resistant	Skeptical; forced by management

11.9 Classify Document (Admin)

File: prompts/classify-document.txt | **Endpoint:** POST /api/admin/classify | **Type:** Direct LLM (GPT-5.1-mini, temp 0) | **AWS:** Classify Lambda (Step Functions)

Purpose: Auto-classifies uploaded documents into a content type and generates a summary. Uses cheaper model.

Input: Document name, source label, first 2000 chars. Output: JSON {content_type, summary}.

Types: case_study, transcript, blog_post, journal_article, report, guide, policy_brief, lecture, tool_or_resource, other.

12. Environment Variables

Variable	Req	Description	Default
WEAVIATE_HOST	Yes	Weaviate cluster hostname or URL	localhost:8080
WEAVIATE_API_KEY	Yes*	Weaviate API key (* not needed for local Docker)	(none)
WEAVIATE_SCHEME	No	http or https	https
OPENAI_API_KEY	Yes	OpenAI API key for GPT and embeddings	(none)
CHATBOT_MODEL	No	Model name for all agentic prompts	gpt-5.1
CHATBOT_MAX_RESULTS	No	Max search results per KB query	5
PORT	No	Server port	3000

Loaded from .env file via dotenv. Template at .env.example. On Render, configured in render.yaml. On AWS, stored in Secrets Manager and injected into ECS tasks and all Lambda functions at runtime.

Generated March 2026 – Coaching Tool for Public Engagement